Téma 1: Kompilace, interpretace a verzování kódu

Teoretický základ

Než začneme programovat, musíme pochopit, jak počítač našemu kódu rozumí a jak kód bezpečně uchovávat.

- **Kompilovaný jazyk (např. C++):** Celý zdrojový kód se nejprve přeloží (zkompiluje) pomocí kompilátoru do strojového kódu (soubor .exe). Běh je pak velmi rychlý, ale kompilace trvá.
- **Interpretovaný jazyk (např. Python):** Kód se nekompiluje předem. Speciální program (tzv. interpret) bere kód řádek po řádku, překládá ho a rovnou spouští. Je to pomalejší, ale flexibilnější pro vývoj.
- **Verzovací systém Git:** Systém pro sledování změn v souborech. Pomáhá týmům spolupracovat a uchovává historii.
  - `git init` - založení lokálního repozitáře.
  - `git add .` - přidání změn do přípravné fáze (Stage).
  - `git commit -m "zpráva"` - potvrzení a zapsání změn do historie.
  - `git push / git pull` - odeslání / stažení změn na vzdálený server (např. GitHub).

Základní koncept k zamyšlení: Merge konflikt

Představte si, že pracujete s kolegou na stejném souboru s kódem. Vy upravíte řádek 10, kolega upraví řádek 10 úplně jinak. Oba se pokusíte odeslat změny na server. V tu chvíli vznikne tzv. **Merge konflikt**, protože Git neví, čí verze je správná. Konflikty se řeší ručně – vývojář musí otevřít soubor, vybrat správnou variantu a provést nový slučovací (merge) commit.

Úkol 1 (Lehká obtížnost)

Zadání úkolu: Vytvořte na svém počítači složku `moje_hra`. Otevřete v ní terminál a udělejte z ní Git repozitář. Vytvořte soubor `main.py`, přidejte ho do fáze stage a vytvořte první commit.

Řešení úkolu 1

```
# Příkazy pro terminál:
mkdir moje_hra
cd moje_hra
git init                    # Inicializace lokálního Gitu
touch main.py               # Vytvoření prázdného souboru main.py
git add main.py             # Přidání souboru do stage
git commit -m "Init: Vytvoření main.py" # Commit změn s popiskem
```

Úkol 2 (Střední obtížnost)

Zadání úkolu: Zjistěte, k čemu slouží soubor `.gitignore`. Vytvořte jej a nastavte tak, aby Git záměrně ignoroval všechny soubory s koncovkou `.log`. Následně tento stav uložte (commitněte).

Řešení úkolu 2

```
# Soubor .gitignore říká Gitu, které soubory nemá vůbec sledovat (např. tajná hesla).
touch .gitignore
echo "*.log" > .gitignore # Zápis pravidla do souboru (ignoruj všechny logy)
git add .gitignore
git commit -m "Přidání ignorování logů"
```

Úkol 3 (Těžká obtížnost)

Zadání úkolu: Propojte svůj lokální repozitář s prázdným repozitářem na GitHubu pomocí příkazu `git remote add origin [URL]` a odešlete (pushněte) svůj první commit do větve `main`.

Řešení úkolu 3

```
# Propojení s existujícím prázdným repozitářem na GitHubu
git remote add origin https://github.com/vase-jmeno/moje_hra.git

# Přejmenování hlavní větve na moderní standard 'main'
git branch -M main

# Odeslání lokálních commitů na vzdálený server (push)
git push -u origin main
```

Úkol 4 (Lehká obtížnost - dodatek)

Zadání úkolu: Zjistěte aktuální stav vašeho repozitáře (které soubory byly změněny, jaké čekají na uložení). Následně si vypište celou historii vašich dosavadních commitů.

Řešení úkolu 4

```
# Zjištění aktuálního stavu repozitáře (např. sledované vs. nesledované soubory)
git status

# Výpis historie (seznam commitů s jejich ID, autorem a časem)
git log
```

Úkol 5 (Střední obtížnost - dodatek)

Zadání úkolu: V souboru `main.py` jste udělali několik špatných úprav, které chcete zahodit (vrátit soubor do stavu, v jakém byl při úplně posledním úspěšném commitu). Změny jste naštěstí ještě nepřidali do stage (přes `git add`). Jak příkazem vrátíte soubor zpět?

Řešení úkolu 5

```
# V moderním Gitu zahodíte lokální úpravy souboru pomocí restore:
git restore main.py

# (V některých starších systémech nebo materiálech uvidíte také tento příkaz:)
# git checkout -- main.py
```

Úkol 6 (Těžká obtížnost - dodatek)

Zadání úkolu: Vytvořte v repozitáři novou boční vývojovou větev s názvem `testovani`, přepněte se do ní, představte si, že jste vytvořili commit a následně tuto novou větev slučte (merge) zpět do vaší hlavní větve (`main`).

Řešení úkolu 6

```
# 1. Založení nové větve a okamžité přepnutí do ní
git checkout -b testovani

# -- V této chvíli byste normálně pracovali a dělali commity --

# 2. Jakmile je hotovo, přepneme se zpět do hlavní větve
git checkout main

# 3. Sloučení hotové práce z testovací větve k nám do main
git merge testovani
```